

Radio astronomy with Raspberry Pi

Glen Langston | August 31, 2020

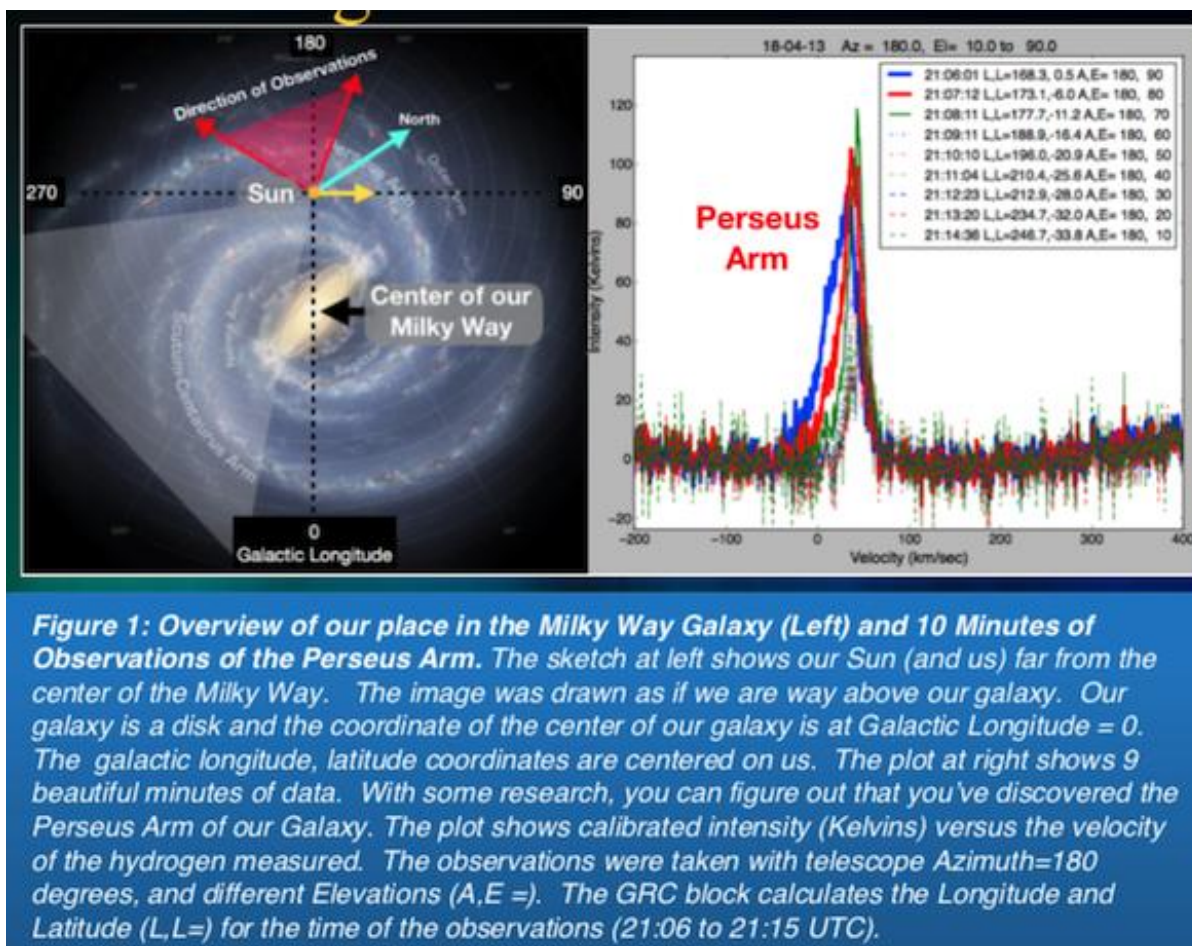
Historical setup guide. Check the current DSPIRA software guide before using older operating system images.

This document describes the Operating System provided by our research group for Raspberry Pis computers configured for Science Aficionado Radio Astronomy. The Operating System (OS) is courtesy of nearly countless volunteers in the Raspberry Pi Foundation, the Ubuntu Organization, the [GnuRadio Collaboration](#) and the folks at SDRplay. The GnuRadio Collaboration made home radio astronomy possible, thanks to their free software for digital signal processing. The community of volunteers have been very helpful in creating a fun to use package for learning the fundamentals of modern communications. The SDRplay company put together the basis of the OS package including GnuRadio, in order to ease testing of their RSP1A for numerous applications. The notes here describe only my additions and some hints on setting up networks of radio telescopes, with precise time (hopefully with time tags correct to roughly a microsecond).

Purpose

The goal of this project is to enable everyone to discover that our Milky Way Galaxy surrounds us and is visible day and night, with your own home-built radio telescope. Figure 1 (below) shows our location in the Milky Way and some calibrated observations, like those you will make. There are several steps involved to get to this point. This document is intended to get you to this level of observations in about a week's work, after you've gathered/constructed all the parts. After you accomplish your first observations, the universe is yours to explore. There is virtually no limit to what you can discover with your radio telescopes, if you choose to expand the basic telescope with your own additions.





System Components

The radio telescope consists of a simple metal object to funnel the radio signals from the sky into the first very low noise amplifier (LNA). After increasing the very faint signal by a factor of about a million, by the first amplifier, the signal is fed to a Software Defined Radio (SDR) device. These devices range in price from about \$30 to \$200. The more expensive ones have many unique capabilities. The total telescope cost will be between \$350 and \$500, depending on the size of the telescope you build and which SDR you buy. All the SDR devices plug into computers via USB. A computer is needed to gather the data into a format for display. You can use your own laptop, but we've found that there is more work involved in getting all the software installed than in building the telescope! Therefore we recommend you buy a dedicated Raspberry Pi 4 computer, with power supply, case and SD card for about \$90. (For the full introductory kit, the cost is a little more). 1

1 <https://www.raspberrypi.org/products/raspberry-pi-4-model-b/>

Figure 2 shows all the telescope hardware you're going to need for your first observations. The picture on the left shows the horn, made out of a 1 foot long, six inch diameter (30x15 cm) stove pipe, with a 8 inch to 6 inch duct reducer (20 to 15 cm). A metal box, with LNA, is attached to the side of the horn. For scale, a carpenter's square is 18 by 24 inches (46 by 60 cm).

Figure 2: Left is the horn. At right is the side view of the horn and the computer box containing the SDR and the Raspberry Pi 4 computer. The PI is powered by the blue ethernet cable, which also connects the PI to the computer network. The PI is in a weatherproof box. More detail on the components of the computer system and amplifiers is given in the Appendix.

Software Defined Radio

A Software Defined Radio (SDR) is an electronic device that includes amplifiers and a high speed sampler that measures the incoming signal millions of times a second. The prices of these fantastic devices have dropped dramatically. This document is targeted towards the minimum cost radio telescope and for that system the SDR to use is the \$38 NeSDR, . Figure 3, at right, shows two SDRs attached to one 2

Raspberry Pi computer. Only one SDR is needed. The devices are about 2 cm square and 5 cm long.

Figure 3: Two low cost SDRs attached to a Raspberry Pi computer. The NeSDR comes with antennas that you can use for testing (and playing). However you'll need to construct the special purpose feed for observations of the Milky Way (see Appendix).

2 <https://www.nooelec.com/store/nesdr-smart.html>



Many other SDRs provide more capabilities. The software provided here also works with the Airspy SDRs (mini and R2) , the Analog Devices PlutoSDR and the SDRPlay devices. Each 3 4 5

of these devices has different, fantastic features and additional costs. I've had reasonably good luck with the [Airspy-mini](#) , as it has a flat band pass response, wider 6

bandwidth (6 MHz) and plenty of Bias-tee power for the first LNA. Two amplifiers are needed to use this SDR. I have seen some internal RFI with these SDRs, so take care with shielding. Currently I'm mostly using the SDRplay RSP1A devices, which have wider bandwidth (up to 10 MHz, but I'm using 8.0 MHz), and very good software support. Unfortunately the RSP1A does not have sufficient power in their bias tee circuit for more than one amplifier, so an additional component, a bias tee may be needed. The RSP1A has good shielding, built-in filtering and very high gain. Only one GPIOLabs HI LNA is needed to use this SDR. The RSP1A can run at 10 MHz, but Raspberry Pis can't keep up with that data rate. Alternate source for the RSP1A . 7

Critical First LNA

The first Very Low Noise Amplifier is a critical part of all radio telescopes. The amplifiers used at observatories are often cooled to extremely low temperatures (< 33 Kelvins or -400 Fahrenheit). We'll have to use simpler amplifiers, but fortunately new and improved amplifiers are available with amazing performance at pretty low cost. A short list of amplifiers and places to get them is below, in order of first is best. The amplifiers have two important features, high gain, > 30 dB (a factor of 1000), and very low equivalent noise temperature . Lower noise temperature is better.

1. Prof. Kevin Bandura's custom HI amplifier has the highest sensitivity, but you have to build it yourself. The performance is excellent . The measured noise temperature is 8

around 90 Kelvins. 2. Nick, at GPIO Labs in Canada, has been working with us, and designed a lower cost amplifier for Neutral Hydrogen, with very good performance. The measured noise temperature is around 120 Kelvins. 3. NooElec Neutral Hydrogen Amplifier has an amplifier designed for Neutral hydrogen 9

observations. This amplifier is less expensive and easier to purchase, but has higher measured noise temperature, greater than 150 Kelvins.

3 <https://airspy.com/> 4<https://www.analog.com/en/design-center/evaluation-hardware-and-software/evaluation-boards-kits/adalm-pluto.html#> 5 <https://www.sdrplay.com/> 6 Supports 3.0 and 6.0 MHz bandwidth. 6.0 MHz is excellent for observations of the Milky Way. 7

<https://www.aliexpress.com/item/4000317752046.html?spm=a2g0s.9042311.0.0.74f04c4dwK4pHp> 8 <https://wvurail.org/dspira/DetailedLNAInstructions> 9 <https://www.nooelec.com/store/sawbird-h1.html>

Computer System Setup

This guide assumes you have one or more Raspberry Pi 4 computers for your telescopes and a host computer, either Linux Ubuntu on a x86 laptop or a Mac computer. I almost never connect a keyboard and monitor to my Raspberry Pis, unless there is some sort of problem. Rather the host computer runs Virtual Network Connections to the Pis. After installation you will have a complete computing environment. This Includes a python programming system and C++ compiler. The OS includes GnuRadio and gnuradio-companion, an easy to use tool to explore signal processing. GnuRadio can be used without any SDRs. Click this link to see Sophie and Evan give a quick video introduction to GnuRadio, using only a laptop microphone and speakers.

Figure 4: Raspberry Pi 4 in hand.

Download

The single file with the entire OS is available as a link to a very large file: [NsfSdr-20Aug31.img.xz](#) Look in this Shared Folder for the latest version of the OS:

<https://drive.google.com/drive/folders/1t2soWvGlgze7wg-7QGIVgB0m3Q-BR3C-?usp=sharing> This file

is a 2.8 GB compressed version of the full 9.5 GB OS. The OS is fully functional with many standard programs plus configuration for GnuRadio (version 3.7.13.4) and Radio Astronomy software from West Virginia University. (I hope to upgrade GnuRadio soon.) You will need a 16 GB, or larger, SD card to install the OS. This file must be downloaded to your host computer. Transfer will take a while. This software is the product of many hundreds of volunteers. GnuRadio is a great project built on top of the Linux operating system. This particular version was created by the folks at sdrplay.com, then tuned for Radio Astronomy by our research group.



Transfer to SD Card

I use free download of the program Balena Etcher (<https://www.balena.io/etcher/>) to transfer copies of this OS to SD Cards for my telescopes. I usually use 32 GB cards, but 16 GB cards are sufficient, as the OS is only 9.5 GB. One very nice thing about Etcher is that you don't have to first uncompress the file, just start Etcher writing the compressed file to the SD Card. After configuring your own Raspberry Pi changes for your experiment, you can easily remove the SD card and make an archive version using the Linux "dd" command. Before writing the SD image, I try to clean up any unused files and observations first. 10

Initial Configuration

Because I plan to use this OS in Raspberry Pis for Radio Astronomy, the operating system is configured with WIFI and Bluetooth off. In my configuration, I almost never connect a keyboard and terminal to the Raspberry PI, but rather Virtual Network Connection (VNC) into the PI. For this to work, you need to know the default IP address of the PI. For this version the default IP address is 192.168.1.205. You can turn WIFI on after your first login via VNC. This IP address is chosen because it seems to be the default IP address group for un-managed Power over Ethernet (POE) switches. The switch IP address is 192.168.1.1. The 205 part was chosen because this happened to come from the 5th of the telescopes in my network. I've been using 192.168.1.200 as the IP address of the linux host computer that harvests the data from the telescopes and also provides a GPS time reference. I use the free download version of RealVNC (<https://www.realvnc.com/en/>). This enables multiple connections to several Raspberry Pis. Each PI automatically starts a passwordless VNC session. You probably want to give your VNC sessions passwords, but to avoid initial problems I removed the password.

When you start VNC, the VNC session will complain that the session is not secure, because there is no password. The PI also starts a protected “ssh” server, allowing direct login, without the graphical interface. To login, the special word is “H1!drogen”, without the quotes. Use your mouse and keyboard to run various tutorials on the Raspberry pi operating system and configuration. The top line of the PI has a web browser and a terminal. 11

10 The “dd” command: `sudo dd bs=10m count=900 if=/dev/disk3 of=SDRPlay-20Jun28.img 11`
<https://thepihut.com/blogs/raspberry-pi-tutorials/the-raspberry-pi-tutorial-beginners-guide>

Changing the IP address

Changing the IP address is pretty easy, but requires the use of a text editor. There are many editors in the Pi OS, including nano, vi and my favorite, emacs . Nano is probably the easiest to start with. To change the IP address requires a couple steps. The first is to find out what IP address your computer network uses. To do this, check your home computer. On windows, at the command prompt type: `ipconfig` On a mac or linux system type: `ifconfig` Lots of info will be printed. To find your network, look for the four numbers following the word inet. In my case they are inet 192.168.1.99 . Yours will be different. On the Pi you want to change the first Pi numbers to match the three numbers in the 192.168.1 part. To update the Pi IP address you need to be a “super user root”. To do this on the Pi, click the terminal button on top. When the terminal appears type: `sudo su` You are now “root”. Then edit the file `dhcpcd.conf` in the `/etc/` the directory. To do this type: `nano /etc/dhcpcd.conf` Near the bottom of this file you will see this line: `static ip_address=192.168.1.205` Change the first three numbers to match your home computer router numbers. Pick a number not used on your network for the last number. Usually 205 to 250 are free. Then you need to reboot to have this take effect. Type: `shutdown -r now`

Starting Radio Astronomy

Now you can begin Radio Astronomy observations. The SDRplay folks created a list of things to try after the Pi starts up. Much of this has to do with SDRplay documentation. I’ve added some radio astronomy programs at the end of their long list.

Alternatively, you can use a terminal to type commands. This is the old fashioned way of running a computer. (I’m old fashioned.) If you have amplifiers and an NeSDR attached, you need only type two commands: `cd examples python NsfIntegrate24.py` The first command changes to the directory (`cd`) containing the GnuRadio designs for Radio Astronomy. The second command starts a python program. The programs are also available for free from our github site, [West Virginia University Radio Astronomy Instrumentation Lab \(WVURAIL\)](#). These programs have been tested on Linux x86 laptops, Mac computers, Odroid N2 and Raspberry PI (3 and 4) computers. If you have an AIRSPY you’d use `NsfIntegrate25.py` or `NsfIntegrate100.py` for 2.5 MHz or 10 MHz bandwidth observations. If you have an AIRSPY-mini you’d use `NsfIntegrate30.py` or `NsfIntegrate60.py` for 3.0 or 6.0 MHz bandwidth observations. For the Pluto SDR you’d use `NsfIntegrate45.py` and for the SDRplay you’d use `NsfIntegrate80.py` or `NsfIntegrate90.py` for 8.0 or 9.0 MHz bandwidth observations. After starting the python program, a graphical user interface appears and lots of messages. If this fails the first time, try it again. It seems that the hardware does not wake up on the very first try, but will be fine thereafter. You will see a display looking like the next figure.



Figure 5 : Raw Radio Observations of the Sky.

First Observations

When you've completed the [horn construction](#) , you'll need to first look at the ground to 12 calibration the observations. The calibration consists of two observations. The Hot load is the ground, which has a temperature of about 280 Kelvin. You will need to set the elevation to -90 degrees and set the Record mode to "Average". What about a minute. Set the Observing mode to "Hot/Cold", then change the record mode to "Save" to save the Hot load observation. A red line will appear where the average line was. Set the observing mode back to "Survey" and then set the recording mode to "WAIT". The empty sky is very cold, only about 10 Kelvin (-440 F, -260 C). Go out and move the horn to point straight up. Next set the elevation to up (elevation +90 degrees). Set the Record mode to "Average" again. Wait another minute. Then set the observing mode to "Hot/Cold" and set the Record mode to "Save". A blue line will appear where the average plot was. You've completed calibration! The blue line will show a very weak bump in the plot, which is our Milky Way Galaxy. This can be much more clearly seen in the calibrated plot. Switch the Units to "Kelvins" and the program computes the calibrated spectrum. The broad peak at 1,420.4 MHz is the signal from the spiral arms of our Galaxy. Your plot should look similar to the next figure.

12 More Horn Construction documents are needed. The WVURAIL group has also put together a good construction guide for a different shaped horn. For a classroom construction guide see <https://wvurail.org/dspira/>

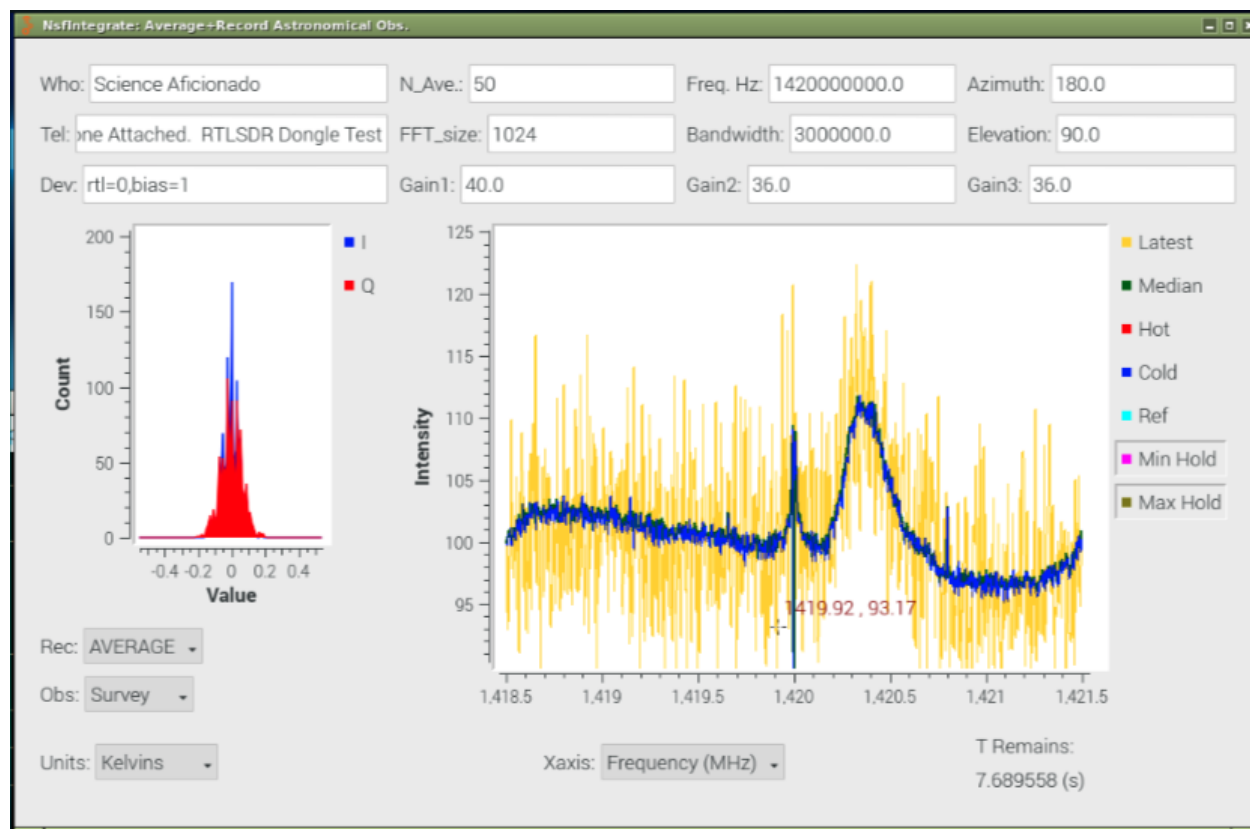


Figure 6 : Calibrated Radio Observations of the Sky. The Milky Way galaxy spiral arms are seen as the broad peak between 1420.2 and 1420.6 MHz.

NTP Server

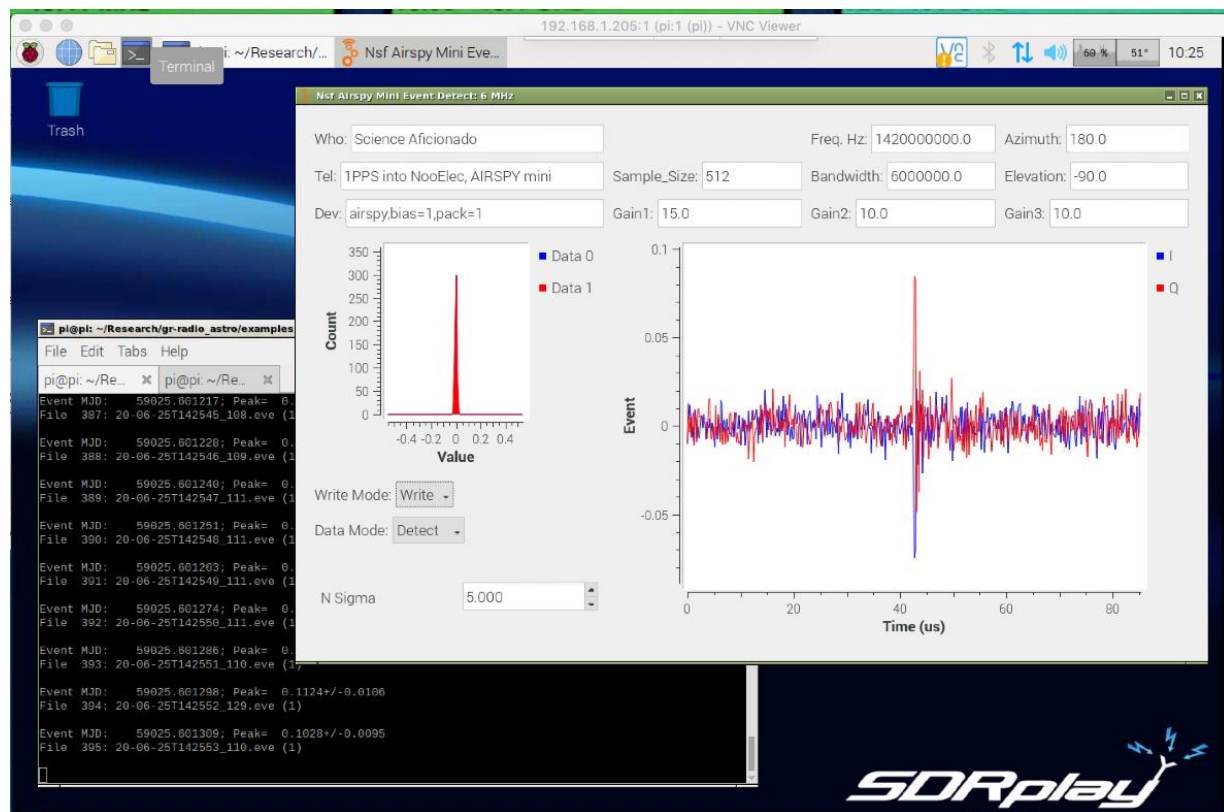
Because the time is critical to Astronomical Observations, it is very important to set the time correctly. Setting the computer clocks after power outages is tedious, so everyone should get a fairly low cost (< \$35) GPS hat (or USB mouse) to attach to some computer in their network. The IP address of a computer is fixed by editing the file “dhcpd.conf” in the /etc directory. You will also want to edit the “/etc/hosts” file to give your telescopes some names. The PI also starts a GPS demon, the Precision Time Protocol demon and the Chrony version of the Network Time Protocol (NTP). You will have to change the IP addresses to set up the time for your network.

Figure 7: Image of the OS with radio astronomy software running a test capture of a GPS 1 Pulse Per Second signal Event. The goal of this software release is to enable an easy introduction to Radio Astronomy and start detecting cosmic ray events. FYI, I use “chrony” as the time control software as it is advertised to provide smoother time transitions. If I understand NTP correctly, it changes only the time, but does not correct the reference oscillator that feeds the clock. Chrony changes the time only once, then only tweaks the rate of the reference oscillator, to keep the time tracking accurately. Note that I have not yet been able to confirm a significant difference between time accuracy between using the “ntpd” demon and the “chrony” demon.

Event Detection

It is our intention that everyone can become a Science Aficionado, by joining a world-wide network of radio telescopes that monitor the sky for cosmic ray flashes. A major part of our software work has been perfecting detection of very short duration (millionths of a second) bright radio flashes when

cosmic rays hit the Earth's atmosphere. These cosmic rays have energies much, much higher than the highest energy particles that can be created in the world's foremost physics laboratories.



Each Aficionado can make a unique contribution, because the flashes extend only a hundred meters on a side, so unless a radio telescope is at that location, the event is missed. The Raspberry Pi 4 designs have been tested with Event capture. The test was performed by feeding a 1 Pulse-Per-Second (1PPS) output from a GPS receiver into a NooElec Neutral Hydrogen amplifier. The early, barebones, version of this amplifier has a control port which is 13

intended to be used for calibration. When the control signal is high, the input is grounding, causing a power change. The 1PPS is a few millions of a second long (a few microseconds). These pulses show up as flashing radio signals in the data stream. These flashes occur with great timing precision, enabling tests of the timing accuracy of the event captures software. The event detection python programs have names beginning with "NsfDetect??".py. The last two characters are the sampler clock rate. For example the SDRplay design, sampling at 9MHz has "NsfDetect90.py". When running on a Raspberry Pi 4 the design has been tested, and is found to capture all 600 1 PPS pulses in 10 minutes. The Raspberry Pi 3 computers are slower and only designs working at less than 3 MHz less are able to accurately capture events without losing many samples. In a 10 minute test, with a Raspberry Pi 3, running with an Airspy-mini sampling at 3.0 MHz sampling, is able to gather about 75% (484 out of 600) 1 PPS samples. With the Raspberry Pi 4 and an RTL-SDR Dongle, sampling at 2.4 MHz, in a 10 minute sampling of 1 PPS events, all 600 events were detected. The event detection system is still under development and collaborators are welcomed.

Summary

You have the opportunity to discover the Milky Way for yourself, with the tools provided. These tools are the door to a marvelous, unseen universe. Join the collaboration and make your own contributions!

LightWork Memos

The more detailed tests of the system are in a set of memos. You're invited to contribute to these memos: <https://wvurail.org/lightwork/>

13 https://www.amazon.com/Nooelec-SAWbird-H1-Barebones-Applications/dp/B07XJLKQDN/ref=sr_1_8?dc_hild=1&keywords=nooelec+amplifier&qid=1593445896&sr=8-8

Live Video Demo

We gave a live demo of the current horn design in an online webinar during the American Astronomical Society Meeting in June 2020. The demo shows the telescope in operation, easily finding our Milky Way Galaxy spiral arms in a few seconds.

<https://www.gb.nrao.edu/~glangsto/aas2020/Langston2020AasTelescopeWebinar.mp4> Thanks to Katherine LaFleur for pointing the telescope at the Milky Way!

Appendix 1: The complete amplifier and computer system

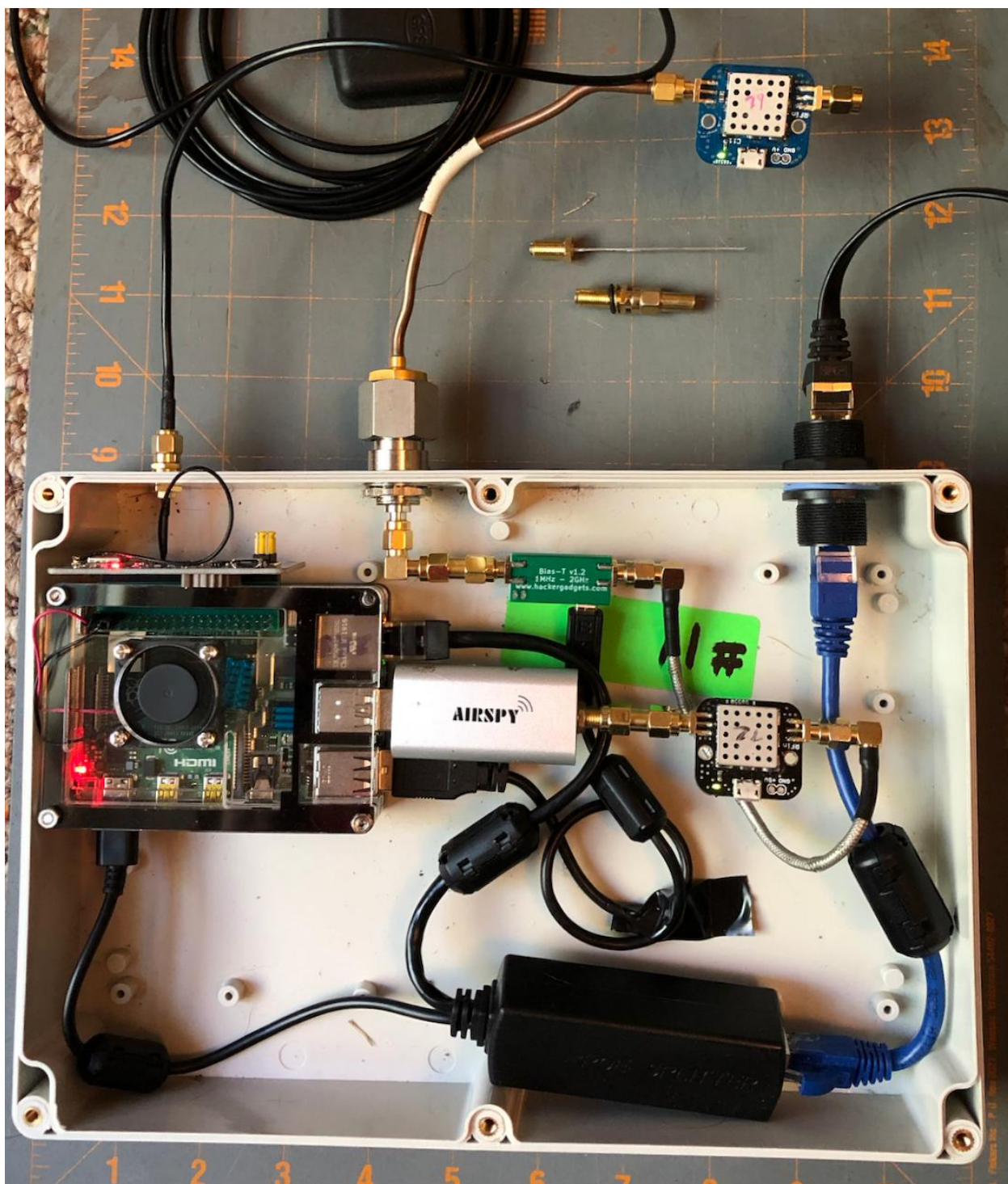
The complete system for full time observations has several components. My intention was that each and every student/teacher/hobbyist/aficionado could contribute to observations of the sky and transient event detection. To achieve this, the computers need to be in weather proof boxes. The system also needs lots of gain from low noise amplifiers. Accurate timing is needed for precise event detection. We're still working on getting the event detection time precision to better than a millionth of a second. Below we describe a system that places all computing components inside a watertight box, intended to be placed under the horn telescope. This system includes two GPIO Labs Neutral hydrogen amplifiers , an AIRSPY mini , a GPS hat , a Power Over Ethernet splitter, a bias-tee 14 15 16

and Raspberry pi case with fan. The assembly of these components is described elsewhere. My current favorite system includes only one GPIO Labs Neutral hydrogen amplifier, a SDRPlay RSP1A, the GPS hat, a Power Over Ethernet "hat" with fan (not a splitter), a bias-tee and Raspberry Pi 4 with metal standoffs. The computer and amplifier are connected directly to the horn in metal boxes. The assembly of these components will also be described elsewhere. To test proper operations, the 50 Ohm Load is also needed. This 50 Ohm noise source input 17

provides power than all the power the horn will collect! You can certainly improve on these designs. Give one a try, then improve your telescope.

14 GPIO Labs - <https://gpio.com/> 15 AIRSPY - <https://airspy.com/airspy-mini/> 16 Waveshare: <https://www.waveshare.com/catalogsearch/result/?q=MAX-M8Q+GNSS+HAT>

17 https://www.amazon.com/DHT-Electronics-coaxial-connector-Termination/dp/B00BXUYDMM/ref=sr_1_6?crid=OMX57J9QT9LU&dchild=1&keywords=50+ohm+load+sma&qid=1598708556&sprefix=50+ohm+load+caps%2C145&sr=8-6

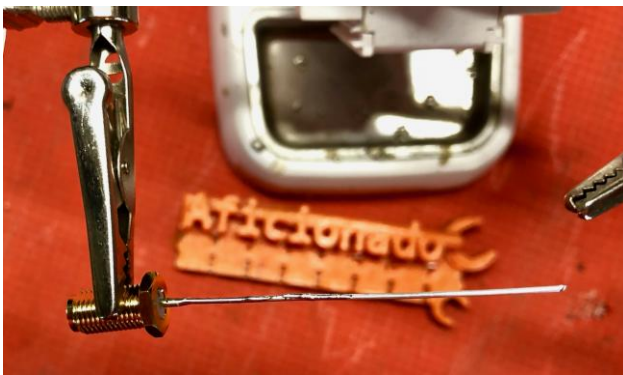


Appendix Figure 1: Telescope Observing system, including a Raspberry PI 4, an AIRSPY Mini and two GPIO Labs Neutral Hydrogen amplifiers with input bias tees. The power is fed to the amplifiers on the same coaxial cables as the radio frequency signals. The AIRSPY provides power to the one amplifier and the bias tee powers the other amplifier. The GPS “hat” comes with the bulkhead cable to allow weather proofing. The power comes into the box on the Ethernet cable. The first LNA should be mounted in a weatherproof metal box on the horn..

The input of the first LNA is the SMA bulkhead connector with a 5 cm silver wire soldered on. This is the only soldering needed for this project.

Appendix Figure 2: Closeup of the GPIO Labs amplifier, the input feed probe and 50 Ohm Load. The background has 1 inch square lines. A long coaxial cable is needed to connect this amplifier to the computer box. I prefer thicker, stronger, N connectors for making this connection.

Appendix Figure 3: I created a 3d printed ruler for measuring and cutting 5cm of the silver wire. A hole in the centimeter ruler holds the wire when it is being cut. The ruler is 8 cm long because that is the distance from the end of the horn to the feed probe hole. The soldering is about as easy as can be, if you've got a stand to clamp the parts. The wire should be held with paper to keep from heating up the stand while soldering.



Appendix Figure 4: Left shows the SMA connector and wire before soldering. After soldering the feed probe is held by the SMA connector bolted to the horn. For the first 50 people reading to this point, email me and I'll send you a soldered feed probe! - glen.i.langston@gmail.com





Appendix 2: Gparted

The SD card can be used immediately after the OS is transferred. However the OS was compressed to make downloading quicker. If you want to use the full sized SD card, the card needs to be re-partitioned after writing. (First test the SD card OS in your Pi, then expand later if needed.) On the OS there is a Linux program, `gparted`, to increase the partition to full size. Using the linux “`gparted`” command (available via `sudo apt-get install gparted`), I plug the SD card into a USB adapter and increase the partition to the full size of the SD card. After getting the Radio Astronomy software working (to the best of my ability), I then “`gparted`” it back down to a small size before saving it for distribution.